

tf.compat.v1.feature_column.categorical_column_with_vocabulary_file

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/feature_column/categorical_column_with_vocabulary_file

A CategoricalColumn with a vocabulary file. (deprecated)

Function Signatures

```
tf.compat.v1.feature_column.categorical_column_with_vocabulary_file(
    key, vocabulary_file, vocabulary_size=None,
    num_oov_buckets=0, default_value=None, dtype=tf.dtypes.string
)
```

Code Examples

```
tf.compat.v1.feature_column.categorical_column_with_vocabulary_file(
    (
        key,
        vocabulary_file,
        vocabulary_size=None,
        num_oov_buckets=0,
        default_value=None,
        dtype=tf.dtypes.string
    )
)
```

```
tf.compat.v1.feature_column.categorical_column_with_vocabulary_file(
    key,
    vocabulary_file,
    vocabulary_size=None,
    num_oov_buckets=0,
    default_value=None,
    dtype=tf.dtypes.string
)
```

```
tf.dtypes.string
```

```
import tensorflow as tf
states = tf.feature_column.categorical_column_with_vocabulary_file(
    key='states', vocabulary_file='states.txt', vocabulary_size=5,
    num_oov_buckets=1)
columns = [states]
features = {'states':tf.constant([[ 'california', 'georgia',
' michigan',
'texas', 'new york'], ['new york', 'georgia', 'california',
' michigan',
'texas']])}]
linear_prediction =
tf.compat.v1.feature_column.linear_model(features,
columns)
```

```
import tensorflow as tf
states = tf.feature_column.categorical_column_with_vocabulary_file(
    key='states', vocabulary_file='states.txt', vocabulary_size=5,
    num_oov_buckets=1)
columns = [states]
features = {'states':tf.constant([[ 'california', 'georgia',
    'michigan',
    'texas', 'new york'], [ 'new york', 'georgia', 'california',
    'michigan',
    'texas']])}]
linear_prediction =
tf.compat.v1.feature_column.linear_model(features,
columns)
```

```
import tensorflow as tf
states = tf.feature_column.categorical_column_with_vocabulary_file(
    key='states', vocabulary_file='states.txt', vocabulary_size=6,
    default_value=0)
columns = [states]
features = {'states':tf.constant([[ 'california', 'georgia',
    'michigan',
    'texas', 'new york'], [ 'new york', 'georgia', 'california',
    'michigan',
    'texas']])}]
linear_prediction =
tf.compat.v1.feature_column.linear_model(features,
columns)
```

```

import tensorflow as tf
states = tf.feature_column.categorical_column_with_vocabulary_file(
    key='states', vocabulary_file='states.txt', vocabulary_size=6,
    default_value=0)
columns = [states]
features = {'states':tf.constant([[ 'california', 'georgia',
    'michigan',
    'texas', 'new york'], [ 'new york', 'georgia', 'california',
    'michigan',
    'texas']])}]
linear_prediction =
tf.compat.v1.feature_column.linear_model(features,
columns)

```

```

import tensorflow as tf
states = tf.feature_column.categorical_column_with_vocabulary_file(
    key='states', vocabulary_file='states.txt', vocabulary_size=5,
    num_oov_buckets=1)
columns = [tf.feature_column.embedding_column(states, 3)]
features = {'states':tf.constant([[ 'california', 'georgia',
    'michigan',
    'texas', 'new york'], [ 'new york', 'georgia', 'california',
    'michigan',
    'texas']])}]
input_layer = tf.keras.layers.DenseFeatures(columns)
dense_tensor = input_layer(features)

```

Documentation

A CategoricalColumn with a vocabulary file. (deprecated)

Use this when your inputs are in string or integer format, and you have a vocabulary file that maps each value to an integer ID. By default, out-of-vocabulary values are ignored. Use either (but not both) of `num_oov_buckets` and `default_value` to specify how to include out-of-vocabulary values.

For input dictionary features, `features[key]` is either Tensor or SparseTensor. If Tensor, missing values can be represented by -1 for int and "" for string, which will be dropped by this feature column.

Example with `num_oov_buckets`:

File `/us/states.txt` contains 50 lines, each with a 2-character U.S. state abbreviation. All inputs with values in that file are assigned an ID 0-49, corresponding to its line number. All other values are hashed and assigned an ID 50-54.

Example with `default_value`:

File `/us/states.txt` contains 51 lines - the first line is 'XX', and the other 50 each have a 2-character U.S. state abbreviation. Both a literal 'XX' in input, and other values missing from the file, will be assigned ID 0. All others are assigned the corresponding line number 1-50.

And to make an embedding with either:

Returns

Raises

